Predictive Reasoning System – Practice Module 2 Final Report

Remymy: An Agentic Conversational Meal-Ranking System

Group 44

Members:

Le Chi Thanh - A0352811L

Vuong Quang Viet Tung - A0357416Y

Tran Dinh Gia Khanh - A0359890J

AIS NUS-ISS 2026

Final Report

Remymy: An Agentic Conversational Meal-Ranking System

(Reuse from the proposal)
Remymy is proposed as a model-centered chatbot that helps people decide what to eat without requiring them to manage a detailed nutrition workflow. The system turns a short, natural-language request into structured preferences, retrieves plausible meals, ranks them with a dedicated model, and explains the result in a compact conversational response. The proposal prioritizes a small, fast-to-deploy architecture: a client-side agent, a narrow Model Context Protocol (MCP) server, a hosted ranking service, and a separate database for meal information. The initial deliverable is an end-to-end prototype with an auditable data pipeline and an evaluation plan focused on ranking quality, constraint satisfaction, usefulness, and latency. This project might focus more on the machine learning model style: on how we could organize, build and evaluate a customer-facing machine learning system.

Introduction

(Reuse from the proposal)
Meal decisions are frequent, repetitive, and often made under constraints such as limited time, available ingredients, budget, dietary preferences, or a desire for something familiar, often leading to decision fatigue or monotonous dietary choices. Many food applications expose large catalogs or calorie tables, but they place the burden of translating an informal intention like “something quick, filling, and not too heavy” into filters and comparisons on the user. Remymy addresses this friction through conversation: The user can state a goal, answer one or two clarifying questions when needed, and receive a short ranked list rather than an exhaustive search result.

The project focuses on the model side of this interaction to use language models to interpret intent and coordinate tools, but it should not be the sole source of nutrition facts or the final ranking authority. Structured food composition data will ground nutritional features, while the ranking model will produce a reproducible score over candidate meals. This separation makes the system easier to test, replace, and deploy than a monolithic chatbot. It also creates a clear boundary for safety: Remymy can describe data-backed meal characteristics and preferences, but it is not a medical diagnostic or treatment system.

The central objective is to demonstrate that a lean conversational layer can make a model-based meal ranker easier to use without obscuring how recommendations are produced. The first version will therefore optimize for a small set of high-value flows: preference capture, candidate retrieval, ranking, explanation, and conversational refinement. The proposal treats usability and model quality as linked: a technically accurate ranker is not useful if the agent asks unnecessary questions, violates explicit exclusions, or returns results too slowly.

Literature review

(Reuse from the proposal)

Research on conversational recommender systems (CRSs) shows why dialogue is useful for recommendation. A survey by Gao et al. (2021) organizes the field around preference elicitation, multi-turn strategy, dialogue understanding and generation, exploration versus exploitation, and evaluation. This suggests that Remymy should represent the conversation as an evolving preference state rather than as a single prompt. The agent can ask for only the missing attribute that most reduces uncertainty, for example, preparation time or an ingredient to avoid then passing the updated state to the ranker.

Nutrient-aware recommendation research provides a second design principle: taste, dietary restrictions, and nutritional expectations must be considered together. Yum-Me combines preference profiling with nutrient-aware meal recommendation and shows the value of learning fine-grained preferences rather than relying only on broad user categories (Yang et al., 2017). Remymy will adapt this insight to a text-first setting by collecting lightweight feedback - such as “more like the second option” or “no dairy” - and converting it into features or constraints for subsequent ranking. Constraint handling is especially important for food. Chen et al. (2021) formulate personalized food recommendation as constrained question answering over a food knowledge graph, including explicit requirements, allergies, nutrition needs, negations, and numerical comparisons.

The literature also reveals a gap that this project can address pragmatically. Many research systems depend on specialized datasets, complex user simulators, or tightly coupled end-to-end models. Remymy instead tests whether a lightweight agent-and-ranker split can provide a useful baseline with clear interfaces and measurable behavior. Evaluation will include ranking metrics such as Recall@k and NDCG@k where labels are available, plus constraint-satisfaction rate, response latency, number of clarification turns, and user acceptance or preference feedback. These measures reflect both model quality and the conversational cost of using the system.

Scope of the project

Remymy is an MVP research-and-engineering prototype for conversational meal recommendation. It will support a user who describes a meal intention, optionally supplies constraints, receives a ranked shortlist, and refines the shortlist through follow-up feedback. The system will use USDA FoodData Central as the authoritative source for food composition attributes. The verified USDA download page provides data in CSV and JSON formats, field documentation, and separate releases for Foundation Foods, FNDDS, Branded foods, and a full download. Because FoodData Central is primarily a food-composition resource rather than a complete recipe catalog, a separate meal database will store meal-level entities, ingredient mappings, preparation metadata, and any project-specific labels.

For Remymy, the practical implication is to keep the interaction loop explicit: interpret the user message, update constraints, retrieve candidates, rank them, present a small result set, and use feedback to revise the next request. This loop can be implemented with a client-side agent and a small number of MCP tools rather than a large orchestration platform.

In-scope

- Data ingestion and normalization for a deliberately bounded subset of FoodData Central, with provenance, FDC identifiers, nutrient fields, units, and release metadata retained.

- A meal database that links meals to ingredients and normalized food records, and stores meal attributes such as cuisine or style, preparation time, serving information, dietary tags, and text descriptions.
- A baseline ranking model that combines hard constraint filtering with a transparent scoring or learning-to-rank component. Candidate features may include nutrient values, ingredient overlap, preparation time, meal tags, and explicit user preferences.
- A small hosted inference service that accepts a structured preference state and candidate meals and returns ranked meal IDs, scores, and machine-readable reason codes.
- A client-side agent implemented with pi-coding-agent in provider mode. The agent will manage the conversational state, decide when to clarify, call the MCP tools, and turn structured results into user-facing language.
- An MCP server exposing a narrow tool surface, such as search meals, rank meals, retrieve meal details, and explain a ranking. Tool inputs and outputs will be schema-defined and validated.
- Offline evaluation using curated or synthetic preference queries and, where possible, held-out interaction or pairwise-preference labels. The evaluation will report top-k ranking quality, constraint satisfaction, diversity, latency, and failure cases.
- Basic observability: request IDs, model version, data release, tool errors, latency, and the inputs needed to reproduce a ranking. No raw private conversation history is required for the MVP unless a user explicitly opts into feedback collection.

Out-of-scope

- Medical diagnosis, treatment, disease management, or individualized clinical nutrition advice. Remymy will present itself as a meal-discovery aid, not a clinician or dietitian.
- A full recipe-generation platform, grocery purchasing flow, payment, delivery integration, restaurant inventory, or real-time price optimization.
- Voice, image, multimodal pantry recognition, wearable integration, or a mobile-native client.
- A large-scale collaborative-filtering ecosystem, social features, multi-user group meal planning, or long-term behavioral health intervention.
- Training or fine-tuning a foundation language model. The MVP will use an existing provider and concentrate project effort on intent representation, tool contracts, ranking, grounding, and evaluation.
- Guarantees about allergy safety, nutrient sufficiency, or health outcomes. Data coverage, serving assumptions, preparation changes, and cross-contamination risks mean that users must verify critical dietary decisions independently.
- Distributed streaming infrastructure, complex feature stores, or multi-region reliability engineering. These may be considered only after the prototype demonstrates value and measurable load requirements.

Team contributions

The responsibilities below follow the project contribution:

Team member	Student ID	Contribution
Le Chi Thanh	A0352811L	Front-end, data preparation, MCP integration and backend
Vuong Quang Viet Tung	A0357416Y	Machine learning model and model serving
Tran Dinh Gia Khanh	A0359890J	Evaluation service, backend and data preparation

Architecture

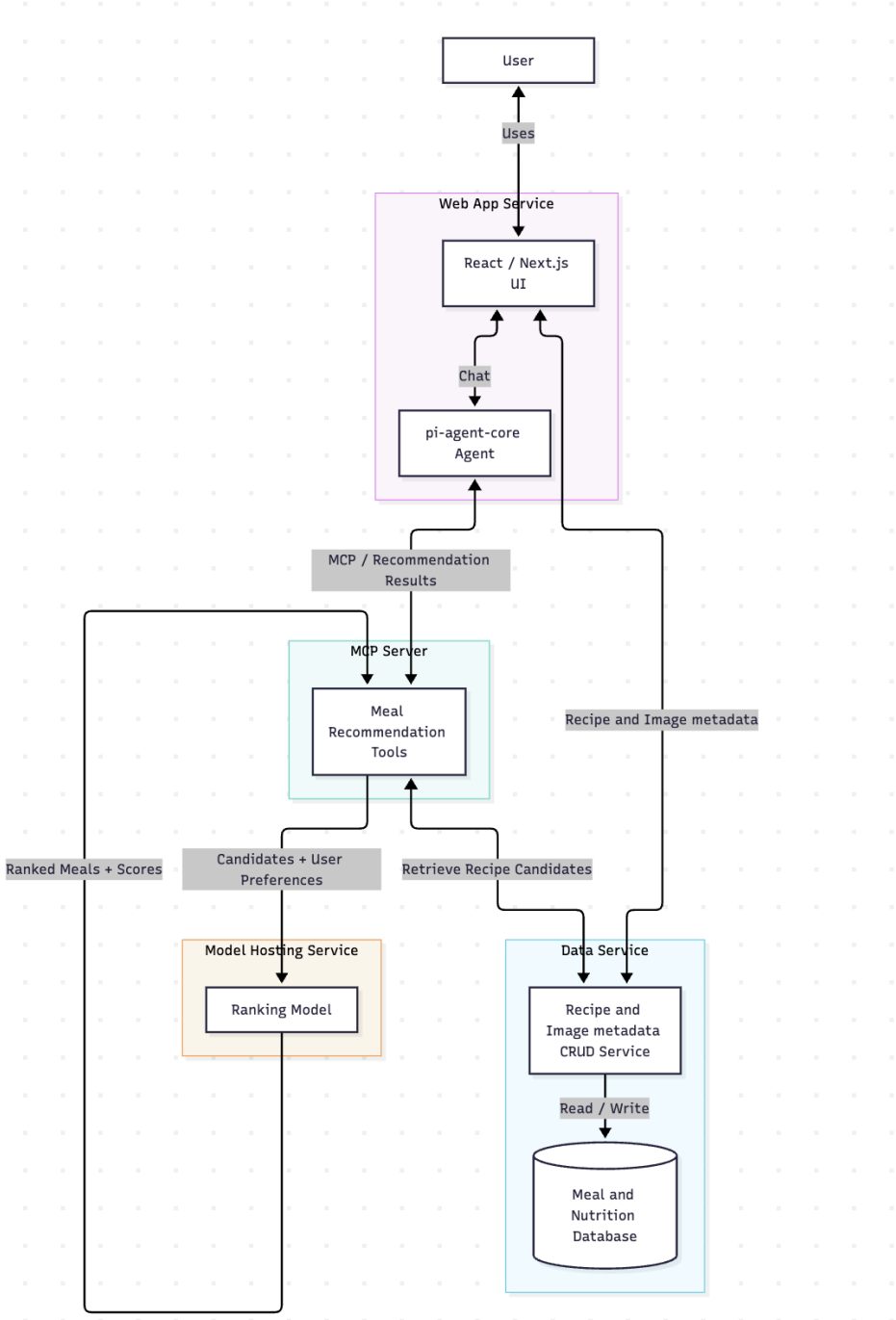


Figure 1: Remymy architecture

In this part, we keep the architecture from our proposal:

The architecture would be organized in a microservice pattern, for detaching a large application into different smaller services. With this direction, there would be 4 main services that we would include in this part, each would be containerized by a separate Docker image:

- A webapp service: Written in ReactJS + NextJS, embedded pi-agent-core (https://github.com/earendil-works/pi/tree/main/packages/agent) as the thin interface for users to interact with the recommendation system.
- An MCP server based on context model protocol v2 as a centralized tool service for the webapp to interact with other services.
- A model hosting service for hosting models, separately
- A CRUD services for retrieving meal data from the things

To justify key architectural decisions, we might have some frequently asked questions:

- Why separate the model hosting endpoint: Technically for the actual scaling solution on production, we often have several replicas of the model serving container waiting to be scaled, in the case there is any peak of the inference requests. Also, the backend requirements of the machine we can use for the web services or the MCP would be much more different than the one hosting model: those API/webapp might just only use the machine which is optimized for CPU-bounded task, while the model might need GPU for running so we think that's better to get them separate.
- Why group the agents onto the frontend app: We might detach the agents onto a remote other services, but for demo purposes (which more focus on how we exploit the methods to create new recommendation models) so we choose to keep the app simple.
- Why detached 2 separate services for MCP and CRUD API:
    - The MCP should be limited on what kind of information they can get from the database since it would fill in the context too much, while in some cases, the prompt injection attack might expose the data from the database, so we should reduce them by having different services.
    - MCP should only focus on the task of feeding what is really needed for a model to answer the questions. In the case of Remymy, the model should only get the ranking result, or the basic info of meals from the database, etc.
    - The CRUD API serves the main usecase of exposing a shared domain interface for other services to get the information of the domain (meals). From our usecases, the MCP should get the data from the CRUD service, and the webapp should call directly to the service to fetch and show the results related to meals with the data from the database.

Machine learning models for recommendation

The project now has recorded experiments for recipe classification, recipe clustering and personalized reranking. The abstraction below preserves the proposed retrieval design; the implemented input/output contracts and experiment results are documented in the following sections

In our discussion, the MVP would be more focused on 1. Define the type of model that we can expose for the AI agents to have the context of the knowledge base that we have 2. What the

machine learning model would look like behind the MCP. There are some of more details we would finalize in our final presentation, but there would be the architecture for our model to push both of the Open Recipes 13k and Food Data Center into a same embedding base for retrieval with the distance-based retrieval or other encode-only architecture for getting the query.

We proposed to use Approximate Nearest Neighbor search (ANN) with Hierarchical navigable small world (HNSW) to retrieve relevant items (using cosine similarity) based on the representative vector of the user's past food ratings compared to semantic values from ingredient names, pricing, and nutrient values. However, since the food history of a user can be diverse and foods can be not similar, we clustered the food history beforehand using Agglomerative Clustering using average-linkage (comparing average embeddings of clusters). The merging will be stopped when the closest pair has the cosine distance above 0.5. For each cluster within the food history, they will be searched with HNSW to retrieve relevant candidates.

After retrieval, candidates are going through a trained reranker that would score how relevant each of these candidates is to the user's past food ratings, dietary preferences and any special needs. The reranker was trained using the Food.com Rating Dataset. For each user in the dataset, a new pair of (user, item) was formed based on items that were rated by that user. For each pair, we build three component vector: Item (using self-attention over every ingredient in a dish alongside its price and nutrient values), User's history (using a windowed Transformer to capture the sequence of rated food), and User's profile (capture preferences and dietary needs semantically alongside with average prices and average nutrient values).

For new users without any history, we relied on Popularity-based Recommendation including filtering (if a user has dietary notes). The recommendation pipeline only starts after there are at least 5 recipes in the history. The number of 5 is also used as a caching threshold for recommendation. The recommendation pipeline is cached for each user and only re-run when they have rated more than 5 recipes since the previous recommendation inference.

Input: candidate ingredients, price and nutrients + the user's dated recipe/rating history and preferences.
Output: P(like) for each candidate.

HNSW retrieval

HNSW is the index that finds the foods most similar to a user's taste. It has a weakness: when a filter (for Halal, vegetarian, etc.) allows only a few foods, it has to explore a large part of the index to find enough of them, which makes it slow. Our system handles diet, allergy and budget constraints in five steps.

1. Tag each food with the diets it fits. Each food is checked for forbidden words in its name and ingredients, for example pork or alcohol for halal. It is tagged with every diet it passes: vegetarian, halal and/or Hinduism.
2. Group foods with similar diet tags. Foods are grouped by how much their diet tags overlap, using agglomerative hierarchical clustering. Each group stores its foods as a

bitmap, a compact list of row numbers. Each group also records what share of its foods has each tag.

3. Pick out the allowed foods for a user. We start with the user's diet and use each group's tag shares. We used :
    a. If no food in a group has the diet (tag share = 0), the whole group is skipped.
    b. If every food has it (tag share = 1), the whole group is kept in one step.
    c. If only some do (tag share > 0), only the matching foods in that group are kept.
4. Foods the user has already rated and foods containing their allergens are then removed. Finally, each remaining food's price is compared to the user's budget, and anything over budget is removed.
5. Choose how to search the allowed foods.
    a. If the allowed foods are 10% or more of the catalog, HNSW searches as normal but may only return allowed foods.
    b. If they are less than 10%, the system compares the user's taste directly against every allowed food instead (an exact scan). Here this took 0.4–0.8 ms, against 3–3.4 ms for filtered HNSW.
6. Cluster user’s food history and search with each cluster to retrieve each set of candidates.
7. Keep matching foods next to each other in RAM. Foods are stored group by group, and cheapest first within each group. So the foods allowed by a diet and budget sit in a few long blocks, which makes the exact scan faster.

General architecture

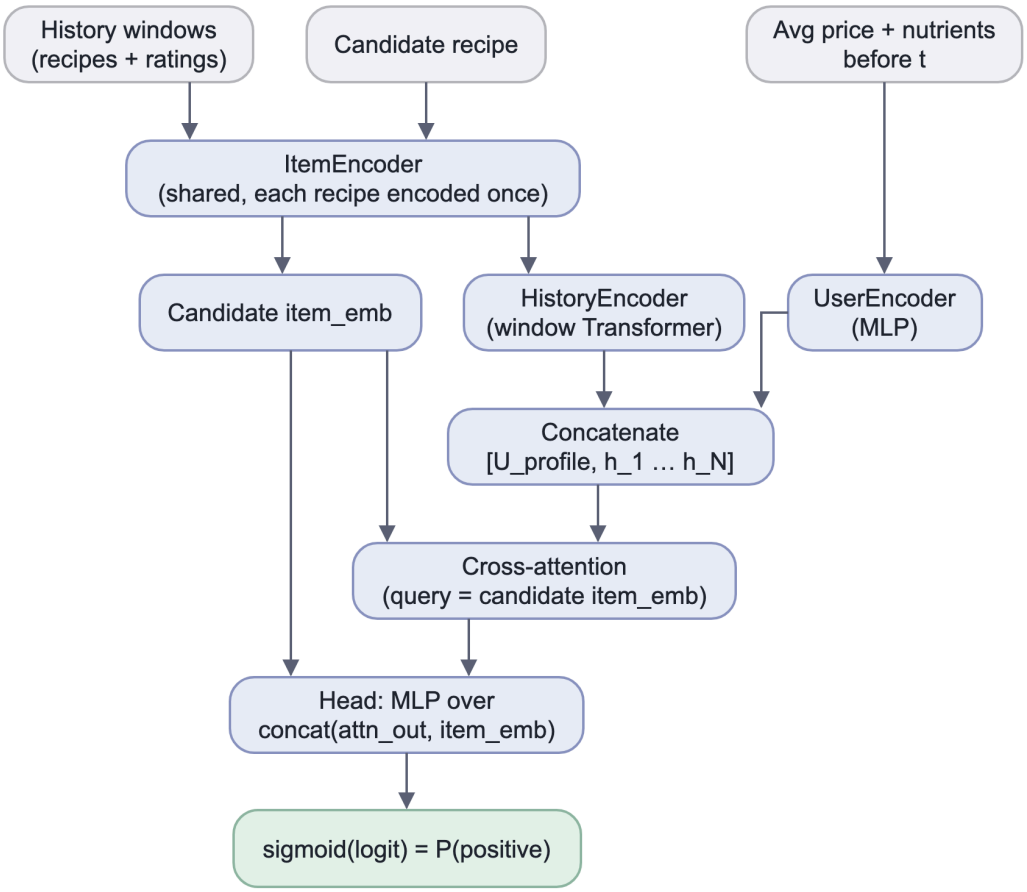


Figure 2: Reranker architecture as proposed. The served model replaces the per-window GRU with a Transformer layer (see Windowed history encoder).

The recommender runs in two stages. The reranker's cross-attention needs one pass for every (user, item) pair, which is too costly to run over all 19,962 items on every request, so retrieval narrows the pool first:

1. **Retrieval.** At startup, every item is embedded once by the ItemEncoder and stored in an HNSW index (cosine distance, M = 16, ef = 200). The query is the mean of the L2-normalized embeddings of the user's 50 newest rated items. The 100 closest items are retrieved.
2. **Reranking.** The reranker scores each of the 100 candidates against the user's full history and outputs P(like), the probability that the user would rate it 3 to 5 stars.

3. **Diversification.** Maximal marginal relevance (MMR, λ = 0.5) picks up to 50 items, each maximizing 0.5 · P(like) − 0.5 · (its highest cosine similarity to an earlier pick). The final list is sorted by P(like).

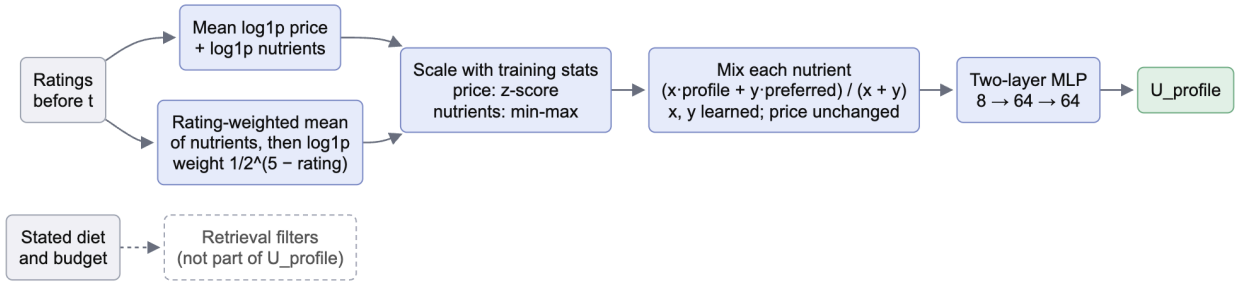
Input for ItemEncoder is based on Content of recipe:

16,878 of the 19,962 items Remymy can recommend (the 10,658 FoodData Central items and 6,220 Open Recipes) were never rated on Food.com, and users can add recipes of their own. An ID-based model, like the matrix factorization and factorization machine baselines in our earlier notebooks, cannot score such items. The reranker therefore sees an item only through its ingredient terms, price and nutrients, and a user only through their rating history, preferences, and special dietary needs, so it can score any item for any user without retraining.

Each ingredient is described by three lists of terms extracted with GLiNER: products (what it is, e.g. "chicken breast"), adjectives ("fresh", "boneless") and verbs ("chopped", "frozen"). The terms are lemmatized and singularized into up to 40 (token, field) pairs from a vocabulary of 2,562 tokens. Token and field embeddings pass through one self-attention layer and are mean-pooled. The layer has no positional encoding, because ingredient order carries no meaning. Self-attention lets each term read the others in its item ("sour" next to "cream"), and the field embedding separates "ground" the verb from "ground" in a product name. Price and seven nutrients (calories, total fat, sugar, sodium, protein, saturated fat and carbohydrates) are log1p-transformed, z-scored and projected to 64 dimensions, and the text and numeric parts are fused into one 64-d item embedding.

Reranker head. The candidate's item embedding is the query of a 4-head cross-attention whose keys and values are the user's profile token and rating states (next two sections). The attention output is concatenated with the item embedding and passed through an MLP to a logit, and its sigmoid is P(like). Because each candidate is its own query, a curry candidate weighs the curries in the history and a cake candidate the cakes, which a single averaged user vector could not do. The whole model has 272,003 parameters.

User profile



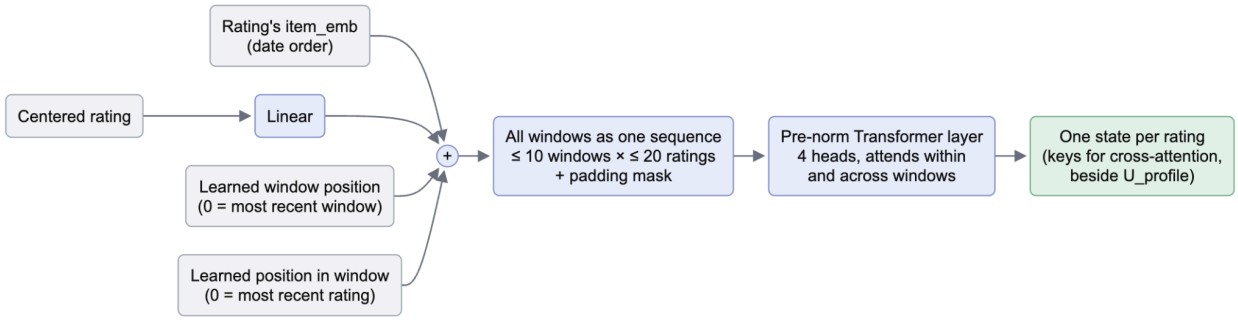
The profile token U_profile summarizes the user's long-run taste in price and nutrition. It is built from two views of the user's ratings before day t:

- **Profile:** The mean log1p [price, 7 nutrients] over those ratings.
- **Preferred nutrients:** For each nutrient, a mean over the ratings with each rating weighted by 1 / 2^(5 − rating): 5 stars counts 1, 4 stars 1/2, down to 1/32 for 0 stars, so liked recipes dominate.

For each nutrient, the UserEncoder mixes the two as (x · profile + y · preferred) / (x + y), with x and y learned, while price passes through unchanged. The two-layer MLP (8 → 64 → 64) then turns the result into U_profile. U_profile sits beside the individual rating states as a key in the cross-attention, marked by a learned type embedding, so a candidate is compared with the user's typical price and nutrition as well as with single past recipes.

Stated preferences. The diet and budget a user sets are not inputs to this part. They act as hard filters during retrieval instead: only items that pass the diet's word screen (vegetarian, halal or Hinduism) and cost no more than the budget can be retrieved, and the budget is checked again on the final picks.

Windowed history encoder



Tastes change over time, so the user's ratings before day t are cut into windows wherever two consecutive ratings are 183 days (about six months) or more apart, making each window one period of taste. We keep the last 10 windows, each with its last 20 ratings. Each rating is centered on the user's mean rating before t and mapped to [0, 1] as (centered + 5) / 10. Centering matters because 74% of the kept ratings are 5 stars, and a 4 from a user who usually gives 5s is a mild dislike. Fixed bounds are used rather than a per-user spread, which would be 0 for a user who rates everything the same.

We used a Transformer-based model for encapsulating and understanding the sequential nature of rating history. Each rating becomes one token: the sum of its item embedding, a projection of its rating, a learned window position (0 = most recent window) and a learned position within the window. The tokens of all windows pass through one pre-norm Transformer layer (4 heads) as a single sequence. This has two advantages over the GRU:

- It keeps one state per rating, so a candidate can attend to a single past recipe rather than to a whole period.
- A rating attends within its own window and across the others, which the per-window GRU could not do.

Training and evaluation

Data. The reranker was trained on Food.com ratings joined to their priced recipes: 1,074,180 ratings by 218,114 users of 218,901 recipes. We kept the 18,888 users with at least 6 ratings, who gave 807,076 ratings, 96.0% of them positive. Ratings of 3 to 5 are positive and 0 to 2 negative. On Food.com, a 0 usually means a review without stars.

Sampled negatives. Each positive is paired with one recipe the user never rated, drawn in proportion to its popularity. We did this because we wanted the model to discriminate between liked and disliked items, not popular and unpopular items. If we just drew randomly on every item, unpopular items (with very few ratings) may be viewed as more disliked. With uniform draws, popular recipes would be the positives and obscure ones the negatives, so popularity alone would separate them. This gives 1,581,504 rows, 49% of them positive.

Split. We split per user and by date: the last rating is the test row, the second-to-last the validation row, and the rest are training rows (1,458,100 / 35,904 / 35,859 rows). A row's history, profile and rating centering use only ratings from days strictly before its own. The vocabulary and normalization statistics are fitted on training rows only.

Optimization. We used AdamW (learning rate 1e-3, weight decay 1e-5) on binary cross-entropy, with gradients clipped at 1.0. Each batch of 512 is built from chunks of up to 16 consecutive rows of one user, which share most of their history. Early stopping watched validation log loss (patience 2, at most 10 epochs). The best epoch was the last, with validation log loss 0.6103 and still falling, and an epoch took about 16 minutes on CPU. For comparison, we trained a "flat" Transformer on the same batches. It reads only the last 20 ratings, using RoPE and grouped-query attention, with attention limited to ratings within 183 days of each other.

Results. For the ranking metrics, each of the 17,059 held-out positives is ranked among 100 candidates: itself plus 99 recipes the user never rated, drawn like the training negatives.

Metric	**Value**
Log loss	0.613
ROC-AUC, all rows	0.718
HR@10	0.319
Mean rank (of 100)	28.5

Table 2: Reranker results on the test split

- **Ranking works.** HR@10 is 3.1 times that of random order and 2.1 times that of popularity.

Limitations.

- The reranker was trained on whole Food.com recipes (nutrients per serving, whole-recipe prices) but also scores FDC items (nutrients per 100 g, single-item prices). P(like) should be read as a ranking signal, not a calibrated probability.

Expected API spec

For the expected API spec for the model service, we would define the schema and the input/output of the model like this:

```
{
  "request_id": "req-123",
  "query": "quick high-protein Indian food",
  "preferences": {
    "diet": "vegetarian",
    "max_minutes": 25,
    "exclude": ["peanuts"],
    "cuisine": "Indian",
    "goal": "high_protein"
  },
  "candidate_ids": ["recipe-12", "recipe-85", "recipe-204"],
  "limit": 6
}
```

Figure 3: Input for the model endpoint

```
{
  "request_id": "req-123",
  "recommendations": [
    {
      "recipe_id": "recipe-85",
      "score": 0.91,
      "reasons": [
        "Matches Indian cuisine",
        "High-protein ingredients",
        "Fits the 25-minute limit"
      ],
      "nutrition_confidence": 0.84
    },
    {
      "recipe_id": "recipe-12",
      "score": 0.76,
      "reasons": [
        "Vegetarian",
        "Similar ingredients to the request"
      ],
      "nutrition_confidence": 0.68
    }
  ],
  "returned_count": 2,
  "model_version": "content-ranker-v1",
  "catalog_version": "open-recipes-13k-v1",
  "mapping_version": "fdc-linker-v1"
}
```


Figure 4: Output for the model endpoint

The implemented endpoint is narrower than this proposal. The client calls `GET /api/recommend/{username}?top_k=10` (top_k from 1 to 50), and the API reads everything else from the database: the user's 50 newest ratings (recipe_id, like, timestamp) and their preferences (special_diet, budget). The response has two fields:

- **profile:** the user's buying profile, with avg_price (SGD) and avg_nutrient_values (per 100 g) averaged over the priced items in the user's rated recipes. It is null when none of them matched a priced item.
- **items:** up to top_k recommendations, most likely to be liked first, each with an id, a title and a score, which is P(like).

A user with no ratings gets an empty list. The reranker reads no free-text query, time limit or exclusion list. Of the stored preferences, only the diet and the budget filter the results: cuisines, allergies and the preferred nutrient are stored but filter nothing.

Expected flow on online recommendation

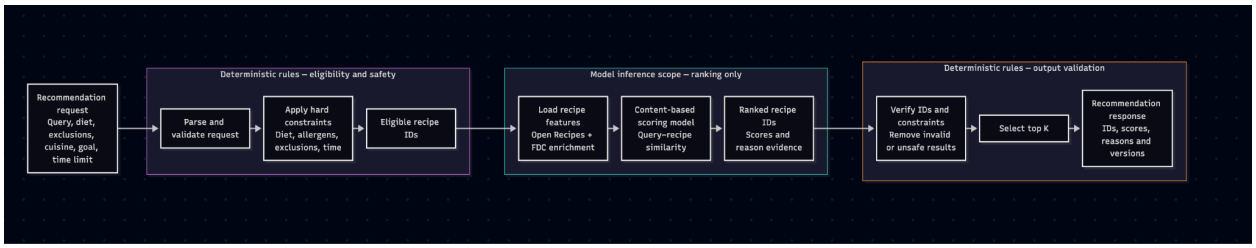


Figure 5: Online inference flow for the MCP.

This is a combination of the real production-grade machine learning system, combined with both deterministic and non-deterministic rules to retrieve relevant records from the database.

For the recommendation space, there might be some questions related to including the LLM as a part of the inferences, but from our side, our tolerance for error is too low to use the LLM for synthesis the data, therefore, we would consider LLM as a human interface for people to interact with the result of the machine learning model we trained. Hence, we would put this out of the chart, but in a later report we would provide the system prompt that we would use to let the model answer the result as we expected.

Inside the recommendation endpoint, one request runs as follows:

1. **Cache.** If Redis holds this user's recommendations, they are returned, cut to top_k.
2. **History.** A rated item that the pipeline holds enters the history as itself. Any other recipe (a user's own, or an Open Recipe) enters as the catalog foods its ingredients match. A like counts as a 5-star rating and a dislike as 0.
3. **Retrieval.** The 100 items nearest to the query vector are retrieved, excluding the history's own items. If the user has set a diet or a budget, only items that meet them can be retrieved.
4. **Near-duplicate filter.** A candidate at cosine 0.97 or more to any history item is dropped, because the catalog holds the same food several times (for example as patties and as crumbles).
5. **Reranking.** The reranker scores the remaining candidates.
6. **Response.** The picks within budget are sorted by P(like), all of them are cached in Redis for one day, and the first top_k are returned. The cache is cleared when the user changes their preferences and on every 11th rating.

Database schema

No change from our side for this part.

The CRUD API reads an immutable SQLite database created by the ingestion bootstrap. The schema preserves source provenance, separates recipe and FoodData Central entities, and versions ingredient matching and derived nutrition.

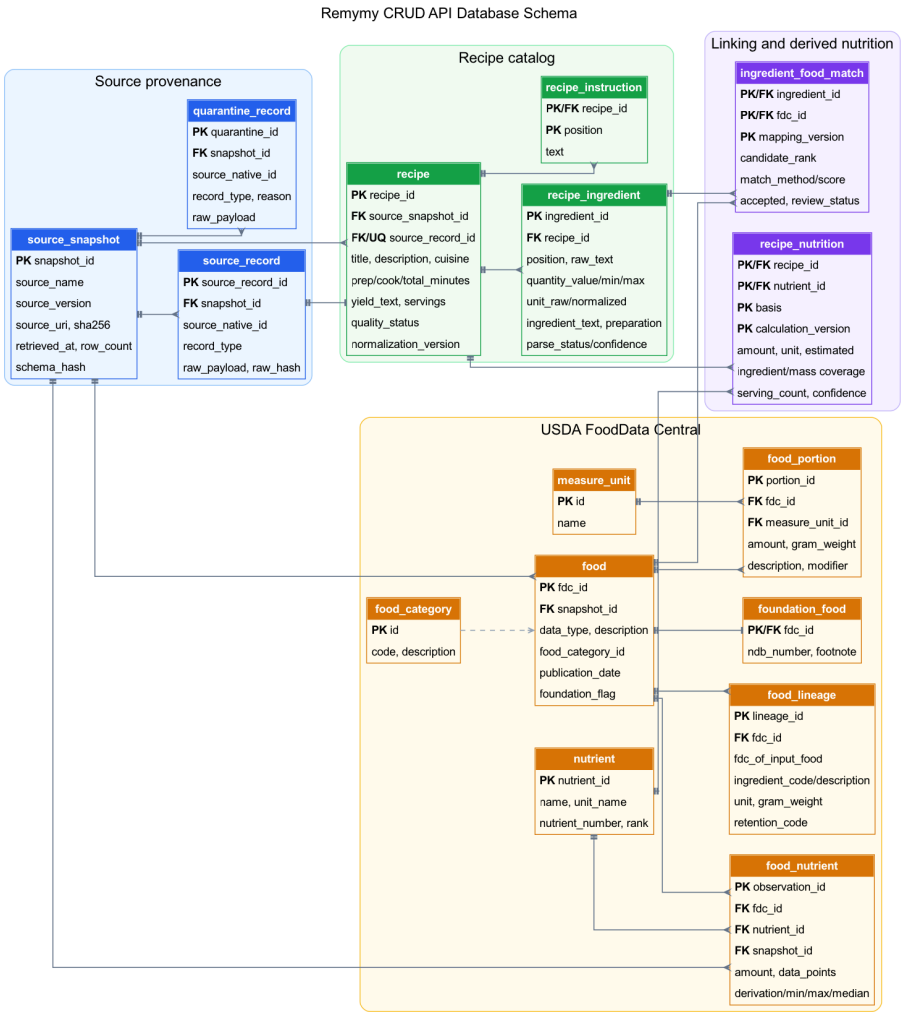


Figure 6: Remymy CRUD API database schema.

Agents in detail proposal

With the agent design, as mentioned above, the agent will be directly integrated into the frontend of web application. For the scope of agents, the team would define the scope of itself, as well as treating the web application as the agent harness itself. To build the agents, we are using the pi agent core (https://github.com/earendil-works/pi) for directly embedding the agents into our TypeScript application.

For the agents, this currently brief system prompt:

> *You are Remymy, a warm and concise meal recommendation assistant.*
> *The only authoritative meals are the curated MEALS returned by the recommend_meals tool.*

Call recommend_meals whenever the user asks for meal ideas, changes preferences, or asks what to eat.Call say_hello whenever the user asks to test MCP or asks you to say hello; it is backed by the Remymy Python MCP service.
The authoritative starting preference state is {preferences JSON}. Preserve its diet, time limit, and exclusions in every tool call.
If the user wants meat-free meals, pass vegetarian or vegan into recommend_meals. Never list meat, poultry, or fish when that constraint is active.
Keep the final answer brief, human, and in simple markdown. Mention only meals returned by the tool.
Do not provide medical or nutrition guarantees; the catalog contains demo estimates.
If you're not sure about anything, or not related scope, please reject the requests.

This is a simple constraint that we provide for guiding the agents to work with our system. The system prompt itself has the practical list of do-and-don't list for guiding what is the scope of the agents:

Do:

- Be warm, concise, and use simple Markdown as the output format and great tone.
- Call recommend_meals for meal ideas, preference changes, or "what to eat" while recommending only meals returned by that tool from the MCP as the single source of truth. This prevents the model from hallucinating, then recommend something outside of the system
- Preserve the user's diet, time limit, and ingredient exclusions.

Don't

- Invent meals outside the tool's results.
- Ignore or relax the user's dietary restrictions.
- Recommend meat, poultry, or fish when meat-free restrictions apply.
- Make medical claims or nutrition guarantees. This is not in the scope of our system and users requiring clinical nutrition plans should be referred to a certified professional.
- Give long, redundant answers.

In this final report and implementation, we centralize the interface that let agents interact with our system by putting the MCP as the main gateway to expose the tools for the system. This would reduce the attack interface if an agent is being injected with malicious prompts.

Tool	**Inputs**	**Purpose / output**	**Agent usage**
`recommend_meals`	preferences; optional query; limit=6 (MCP default).	Filters Food.com recipes by diet, time, exclusions and cuisine. Returns model-ranked meals, details, scores and reasons.	Called directly for meal requests and preference changes. The agent passes preferences and query.

`say_hello`	Optional name; default "there".	Returns a greeting from the Python MCP service to check the connection.	Called directly when the user asks to test MCP or requests a greeting.
`rank_foodcom_meals`	candidate_recipe_ids; optional user_id and content_scores; limit=6.	Ranks already-filtered candidates through the model service's /v1/rank endpoint. Returns ranked results.	Called internally by recommend_meals; not directly registered with the chat agent.

System evaluation metrics

The explicit-label metrics below remain the planned MVP evaluation. The recorded offline notebook experiments are reported separately afterward: catalog rule-label recovery, unsupervised clustering, and Food.com rating-based reranking. Remymy's live interaction metrics still require real exposure and action logs.

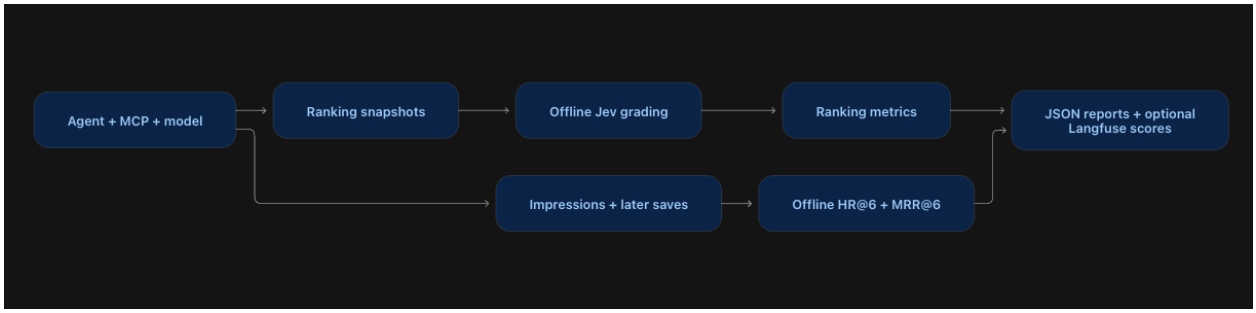


Figure 7: Metric harvester flow

For the harvester flow, we would show the query and candidate recipes in shuffled order, hide the model ranking, and let the evaluator assign grades. Once a query is fully labeled, calculate nDCG@6, Precision@6, Recall@6, and pairwise accuracy for every ranking run attached to it. For behavioral tracking, record impressions when results are actually displayed, then link save/open actions to that request and recipe. Keep browser storage for the Saved UI, but also send those actions to the API. Calculate HR@6 and MRR@6 after the chosen feedback window closes, show them as pending beforehand.

What does change from our previous proposal are:

- We do extend the nDCG, Precision and Recall to at 6, instead of 3. This might cover better for the evaluation for this thing
- The metrics are being evaluated after a conversation turn is done (often being the user -> model loop call tool -> response). After the final result from the agent, we called a hook to let Jev evaluate the past conversation.

For the services, we would use LangFlow (https://github.com/langflow-ai/langflow) for the observability of the whole agentic architecture. The final dashboard for monitoring would cover these bullet points:

- **Explicit quality:** nDCG@6, Precision@6, Recall@6, pairwise accuracy, and labeled-query count.
- **Model comparison:** heuristic versus trained model on the same queries and candidates.
- **Operational health:** constraint violations, empty results, failures, and p50/p95 latency.
- **Feedback:** impressions, saves, and mature HR@6 / MRR@6.

Explicit-label metrics

For each preference query, assign every eligible recipe a human relevance grade g: 0 = irrelevant, 1 = weak, 2 = reasonable, and 3 = best.

- nDCG@6 (primary): gain at rank i = 2^g_i - 1. DCG@6 = sum from i = 1 to 6 of gain_i / log2(i + 1). IDCG@6 is the same calculation for the ideal grade-sorted list. nDCG@6 = DCG@6 / IDCG@6.
- Precision@6: mark a recipe relevant when g >= 2. Precision@6 = number of relevant recipes in the top 6 / min(6, candidate-pool size).
- Recall@6: number of relevant recipes in the top 6 / all relevant recipes in the recorded candidate pool; undefined when the pool contains no relevant recipes.
- Pairwise accuracy: form every labeled pair with different grades. Pairwise accuracy = pairs where the higher-grade recipe is ranked first / all unequal-grade pairs.

Implicit-feedback metrics

Use these only after real exposure and action logs exist. A positive item is a held-out save, meal-plan addition, or cook; an open may be treated as a weaker positive under a frozen labeling rule.

- Hit Rate@6: for each user or evaluation window, score 1 when at least one held-out positive recipe appears in the top 6, otherwise 0. Hit Rate@6 is the mean of this indicator.
- MRR@6: let r be the rank of the first held-out positive recipe within the top 6. Reciprocal rank = 1 / r, or 0 when no positive appears. MRR@6 is the mean reciprocal rank.

Aggregation and decision rule

- Macro-average explicit metrics over preference queries and implicit metrics over users or chronological evaluation windows.
- Compare the model with the current heuristic on the same candidate pool. Select a new model only when nDCG@6 and pairwise accuracy improve without a material Precision@6 regression.

LLM-as-judge and result of evaluation for the whole system

This part is the add-in after our proposal with several changes:

Due to the lack of real human feedback, while trying to imitate the way that big companies are shifting to automate the whole post-training and evaluation system with synthetic feedback, our team explored the solution to rate for the recommendation given. We are using Jev - a Type 1 state-of-the-art decision model. For the cost factor, this is much cheaper for us to run evaluation by batch, while keeping most of the factor of how LLM-as-judge gave us.

The JEV integration is designed to evaluate each interaction through a structured schema containing the model, state, and questions.

The state captures the conversation, user preferences, assistant response, recommended recipe IDs and titles, and tools used, while the questions define the evaluation criteria through yes/no judgments, categorical choices, or rubric-based scores. Once these questions are configured, evaluation can run asynchronously, with the resulting scores attached to the corresponding Langfuse trace. This provides a way to inspect recommendation quality across interactions and identify recurring weaknesses, while human feedback remains necessary to confirm whether the recommendations meet users' expectations.

```
{
  "model": "jev-latest",
  "state": {
    "request": {
      "messages": [
        {"role":"user","text":"Suggest a vegetarian dinner in under 30 minutes."}
      ],
      "preferences": {"diet":"vegetarian","maxMinutes":30,"exclude":
[],"goal":"balanced","cuisine":"any"}
    },
    "response": {
      "text": "Try Chickpea Spinach Curry for dinner.",
      "preferences": {"diet":"vegetarian","maxMinutes":30,"exclude":
[],"goal":"balanced","cuisine":"any"},
      "recommendations": [
        {"id":"demo-001","title":"Chickpea Spinach Curry"}
      ],
      "toolsUsed": ["recommend_meals"]
    }
  },
  "questions": {
    "preference_capture": {
      "type": "noul",
      "instructions": "Do the response preferences preserve the requested vegetarian
diet and a maximum preparation time of 30 minutes?"
    },
    "response_consistency": {
      "type": "noul",
      "instructions": "Does the response text recommend a recipe present in
response.recommendations, without inventing another recipe?"
    }
  }
}
```

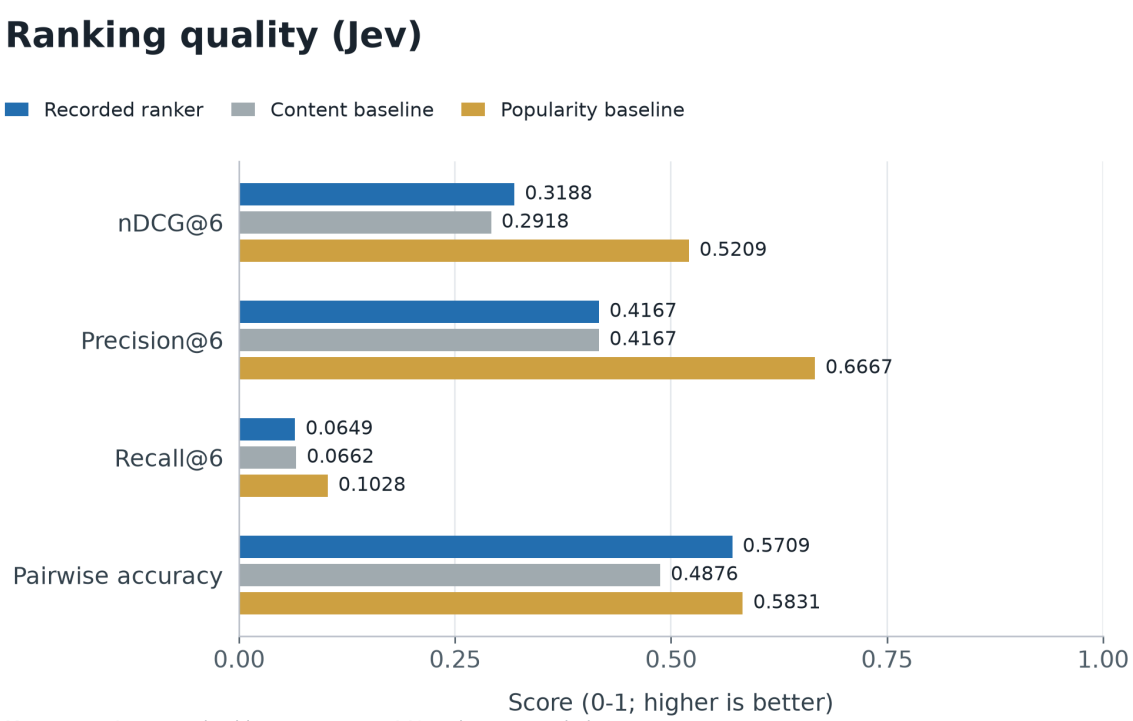

Example JEV input. A synthetic meal interaction is submitted with two explicit evaluation questions.

```
{
  "model": "jev-1.14.0",
  "answers": {
    "preference_capture": {
      "type": "noul",
      "noul": 0.99
    },
    "response_consistency": {
      "type": "noul",
      "noul": 0.98
    }
  },
  "usage": {
    "input_tokens": 527,
    "output_tokens": 42
  }
}
```


Actual Jev output. JEV returned 0.99 for preference capture and 0.98 for response consistency. These are probabilities of a yes answer for this demo, not overall system accuracy.

With the Jev return, the team can calculate the metrics that we mentioned above by:

Metric	**How to calculate from Jev relevance grades**	**Example: [1, 3, 2, 0, 2, 1]**
nDCG@6	Calculate DCG@6 = $\Sigma(2^{g_i} - 1) / \log_2(i + 1)$, then divide by IDCG@6 from the eligible pool sorted by grade. Rewards relevant recipes appearing earlier.	8.4333 / 11.2103 = 0.7523
Precision@6	Treat a recipe as relevant when its JEV grade is at least 2. Divide the number of relevant recipes in the top 6 by 6 (or by the pool size if smaller).	3 relevant recipes / 6 = 0.5000
Recall@6	Divide the number of recipes graded at least 2 in the top 6 by all recipes graded at least 2 in the recorded pool. Undefined if none are relevant.	3 relevant recipes / 3 relevant in pool = 1.0000
Pairwise accuracy	Across all eligible recipes, count unequal-grade pairs where the higher-grade recipe appears first, then divide by all unequal-grade pairs. Ignore ties.	8 of 13 unequal-grade pairs are correct: 8 / 13 = 0.6154



Jev ranking quality on two stored cold-start requests (149 candidate judgments; K = 6). Bars show equal-request means. Small development sample.

Above is the result after we're running batch evaluation for several runs. We can interpret several insight from the Jev results:

- Ordering improves over content, but selection does not. The recorded ranker has higher nDCG@6 (0.3188 versus 0.2918) and pairwise accuracy (0.5709 versus 0.4876), while Precision@6 is tied at 0.4167.
- Popularity remains the stronger reference on this sample. Its nDCG@6 is 0.5209 and Precision@6 is 0.6667, compared with 0.3188 and 0.4167 for the recorded ranker: four versus 2.5 relevant recipes per six, on average.
- Low recall needs its denominator. Recall@6 is 0.0649 within the recorded pools of 49 and 100 candidates. Only six recipes can be selected, so this is not an overall system-accuracy percentage.
- These are two cold-start requests, not 149 independent queries. Balanced held-out requests and human review are needed before generalizing these results to personalized recommendations.

The result is technically showing the way that we can try to improve the system that we have already done:

- Build a stronger evaluation set. Expand beyond the two cold-start requests to cover dietary restrictions, preparation time, budget and follow-up requests. Freeze a held-out query set and identical candidate pools for comparisons, and have independent reviewers grade a sample without seeing the ranking or Jev scores.
- Improve cold-start ranking before adding complexity. Since popularity leads in this sample, test a popularity-based fallback and tune how content, popularity and collaborative scores are combined. Evaluate users with history separately. Promote a change only when it improves held-out nDCG@6 without materially reducing Precision@6 or constraint compliance.
- Strengthen constraint and data validation. Replace substring-only ingredient checks with normalized ingredient and allergen matching, including ambiguous cases such as milk versus soymilk. Carry preparation time, portion basis and price evidence into evaluation; report unknown values separately. Test these checks independently from the ranker so relevance cannot hide a constraint failure.
- Make improvements reproducible and observable. Store model, dataset, judge and ranking-configuration versions with each run. Track end-to-end latency, service errors, empty results and judge cost alongside ranking quality. Cache unchanged judgments and keep offline evaluation separate from live responses. Use these measures to compare a candidate release against the current baseline.

AI Usage

The team used a lot of coding agents (Codex, Claude Code, Kiro) to assist with service development. Each prompt records the service's system instructions, boundaries, and contracts that are carefully being prepared by the team (by hands or just a lot of few-shot prompts to create the prompt), then using the prompt itself to one-shot to get the first version of the service. The prompt itself acted like a contract for the services that if the team wants to change anything in our system then we also have to adjust the prompt. The table below lists the services and their corresponding prompt files:

Service	Purpose	Prompt file
app	Web interface, application routes, and conversational meal agent.	`app.prompt.md`
api	Recipe and user data API, persistence, and recommendation endpoints.	`api.prompt.md`
api-migrate	One-shot database bootstrap and recipe-data migration.	`api-migrate.prompt.md`
mcp	MCP tools for candidate retrieval, constraint filtering, and ranking calls.	`mcp.prompt.md`
models	Serves the trained Food.com hybrid ranking model.	`models.prompt.md`
models-vocab	Downloads the tokenizer vocabulary required by the model service.	`models-vocab.prompt.md`
redis	Shared cache and queue store, with separate API and Langfuse databases.	`redis.prompt.md`
log-collector	Archives container output into hourly log files by service.	`log-collector.prompt.md`
langfuse-web	Trace dashboard and observability API.	`langfuse-web.prompt.md`
langfuse-worker	Processes background tracing jobs.	`langfuse-worker.prompt.md`
langfuse-gateway	Local proxy that opens the Langfuse dashboard using a development session.	`langfuse-gateway.prompt.md`
langfuse-postgres	Stores Langfuse relational metadata.	`langfuse-postgres.prompt.md`
langfuse-clickhouse	Stores analytical trace data for Langfuse.	`langfuse-clickhouse.prompt.md`

langfuse-minio	Provides S3-compatible object storage for Langfuse using RustFS. Fact is the langfuse official guideline using MinIO as main service but MinIO became close-sourced so we have to use RustFS instead.	`langfuse-minio.prompt.md`
langfuse-minio-init	One-shot creation of the Langfuse object-storage bucket.	`langfuse-minio-init.prompt.md`

References

- Chen, Y., Subburathinam, A., Chen, C.-H., & Zaki, M. J. (2021). Personalized food recommendation as constrained question answering over a large-scale food knowledge graph. In Proceedings of the 14th ACM International Conference on Web Search and Data Mining (WSDM '21). ACM. URL: https://doi.org/10.1145/3437963.3441816
- Gao, C., Lei, W., He, X., de Rijke, M., & Chua, T.-S. (2021). Advances and challenges in conversational recommender systems: A survey. arXiv. URL: https://arxiv.org/abs/2101.09459
- Model Context Protocol. (2026). Architecture. URL: https://modelcontextprotocol.io/specification/2026-07-28
- Sun, Y., & Zhang, Y. (2018). Conversational recommender system. In The 41st International ACM SIGIR Conference on Research & Development in Information Retrieval (pp. 235–244). ACM. URL: https://doi.org/10.1145/3209978.3210002
- U.S. Department of Agriculture, Agricultural Research Service. (2019). FoodData Central. URL: - https://fdc.nal.usda.gov/
- Yang, L., Hsieh, C.-K., Yang, H., Pollak, J. P., Dell, N., Belongie, S., Cole, C., & Estrin, D. (2017). Yum-Me: A personalized nutrient-based meal recommender system. ACM Transactions on Information Systems, 36(1), Article 7. URL: https://doi.org/10.1145/3072614